MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Petcov Nicolai FAF-233

Report

Laboratory Work No. 1

Network Programming

HTTP File Server with Docker Implementation

Checked by:
Balan Artiom, *university assistant*
DISA, FCIM, UTM

Chișinău – 2025

1 Objective

Implement a simple HTTP file server using TCP sockets and deploy it using Docker containerization. The server should handle different file types and provide directory listing functionality.

2 Tasks

1. Create a basic HTTP server with TCP sockets that serves files over HTTP
2. Implement support for specific file types: HTML, PNG, and PDF
3. Create a directory listing feature for browsing directories
4. Containerize the application using Docker
5. Create a client for downloading and viewing server content
6. Set up network connectivity for local network access

3 Implementation

3.1 Source Directory Structure

The project directory has the following structure:

```
docker_practice/
          file_server.py          # Main HTTP server implementation
          client.py               # HTTP client for testing
          Dockerfile              # Docker container configuration
          docker-compose.yml      # Docker compose setup
          content/               # Content directory to serve
                index.html         # Main HTML page with image
                books/             # Subdirectory with books
                      programming-guide.pdf
                      networking-book.pdf
                images/            # Images directory
                      library.png    # PNG image file
                readme.txt         # Text file
                sample-document.pdf # Sample PDF file
```

3.2 Docker Configuration Files

3.2.1 Dockerfile

```
FROM python:3.9-slim

WORKDIR /app

COPY file_server.py .
COPY client.py .

RUN mkdir -p /app/content
```

```
9
10 EXPOSE 8080
11
12 CMD ["python", "file_server.py", "/app/content"]
```

3.2.2 Docker Compose File

```
1  services:
2    http-server:
3      build: .
4      ports:
5        - "8080:8080"
6      volumes:
7        - ./content:/app/content
8      networks:
9        - http-network
10
11 networks:
12   http-network:
13     driver: bridge
```

3.3 Server Implementation (file_server.py)

The HTTP file server is implemented in Python using sockets:

```
1  #!/usr/bin/env python3
2
3  import socket
4  import sys
5  import os
6  import urllib.parse
7  from pathlib import Path
8  from datetime import datetime
9
10 def main():
11     if len(sys.argv) != 2:
12         print("Usage: python file_server.py <directory_to_serve>")
13         sys.exit(1)
14
15     serve_directory = sys.argv[1]
16     if not os.path.isdir(serve_directory):
17         print(f"Error: {serve_directory} is not a valid directory")
18         sys.exit(1)
19
20     print(f"Serving files from: {os.path.abspath(serve_directory)}")
21
22     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
23     server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
24
25     host = '0.0.0.0'
26     port = 8080
27     server_socket.bind((host, port))
28
29     print(f"Server starting on {host}:{port}")
30     server_socket.listen(5)
```

```
    print("Server is listening for connections... (Press Ctrl+C to exit)")

    server_socket.settimeout(1.0)  # Set a timeout to make accept() interruptible

    try:
        while True:
            try:
                client_socket, client_address = server_socket.accept()
                print(f"Connection from {client_address}")
                handle_request(client_socket, serve_directory)
            except socket.timeout:
                continue  # Just a timeout, continue the loop
            except Exception as e:
                print(f"Error: {e}")
    except KeyboardInterrupt:
        print("\nServer shutting down...")
    finally:
        server_socket.close()

# Handling file requests
def serve_file(client_socket, requested_path, serve_directory):
    if requested_path.startswith('/'):
        requested_path = requested_path[1:]

    if not requested_path:
        requested_path = '.'

    file_path = os.path.join(serve_directory, requested_path)

    try:
        real_file_path = os.path.realpath(file_path)
        real_serve_dir = os.path.realpath(serve_directory)

        if not real_file_path.startswith(real_serve_dir):
            send_error_response(client_socket, 403, "Forbidden")
            return
    except:
        send_error_response(client_socket, 400, "Bad Request")
        return

    if not os.path.exists(file_path):
        send_error_response(client_socket, 404, "Not Found")
        return

    if os.path.isdir(file_path):
        serve_directory_listing(client_socket, file_path, requested_path)
    else:
        serve_single_file(client_socket, file_path)

# Only supporting HTML, PNG, and PDF files
def get_content_type(file_path):
    extension = os.path.splitext(file_path)[1].lower()

    mime_types = {
        '.html': 'text/html',
```

```
        '.htm': 'text/html',
        '.png': 'image/png',
        '.pdf': 'application/pdf'
    }

    return mime_types.get(extension)
```

3.4 Client Implementation (client.py)

A simple HTTP client was implemented to fetch resources from the server:

```
#!/usr/bin/env python3

import socket
import sys
import os
import urllib.parse

def main():
    if len(sys.argv) != 5:
        print("Usage: python client.py server_host server_port url_path save_directory")
        sys.exit(1)

    server_host = sys.argv[1]
    server_port = int(sys.argv[2])
    url_path = sys.argv[3]
    save_directory = sys.argv[4]

    if not os.path.exists(save_directory):
        os.makedirs(save_directory)

    make_request(server_host, server_port, url_path, save_directory)

def parse_response(response_data, url_path, save_directory):
    # Parse the response headers first
    response_str = response_data.decode('utf-8', errors='ignore')

    header_end = response_str.find('\r\n\r\n')
    if header_end == -1:
        print("Invalid HTTP response")
        return

    headers = response_str[:header_end]
    body_start = header_end + 4

    header_lines = headers.split('\r\n')
    status_line = header_lines[0]
    print(f"Status: {status_line}")

    # Extract HTTP status code
    status_parts = status_line.split(' ', 2)
    if len(status_parts) < 2:
        print("Invalid status line")
        return

    status_code = int(status_parts[1])
```

```
    if status_code != 200:
        print(f"Error: HTTP {status_code}")
        body = response_str[body_start:]
        print(body)
        return

    # Get content type
    content_type = "text/html"
    for line in header_lines[1:]:
        if line.lower().startswith('content-type:'):
            content_type = line.split(':', 1)[1].strip()
            break

    print(f"Content-Type: {content_type}")

    # Handle different file types according to content type
    if content_type.startswith('text/html'):
        # HTML content - print as-is
        body = response_str[body_start:]
        print("HTML Content:")
        print(body)

    elif content_type == 'image/png' or content_type == 'application/pdf':
        # Binary files - save to directory
        body_bytes = response_data[body_start:]
        filename = os.path.basename(url_path) or "downloaded_file"
        if not filename:
            filename = "index.html" if content_type == "text/html" else "file"

        filepath = os.path.join(save_directory, filename)

        with open(filepath, 'wb') as f:
            f.write(body_bytes)

        print(f"File saved to: {filepath}")
        print(f"File type: {content_type}")
        print(f"File size: {len(body_bytes)} bytes")

    else:
        # Unsupported file type
        print(f"Unsupported file type: {content_type}")
        print("Server only supports HTML, PNG, and PDF files.")
```

4 Deployment and Testing

4.1 Starting the Docker Container

To deploy the HTTP server using Docker:

```
# Build and start the container
docker-compose up --build -d

# Verify it's running
docker ps
```

```
# View logs if needed
docker-compose logs http-server
```

The command that runs the server inside the container is:

```
python file_server.py /app/content
```

This is specified in the CMD line of the Dockerfile.

4.2 Content Directory

The server serves files from the content directory, which contains:

```
books/               # Directory containing PDF files
images/              # Directory containing PNG images
index.html           # Main HTML file with links
readme.txt           # Text file (not supported by current server)
sample-document.pdf  # Sample PDF document
```

4.3 Browser Testing

Four different types of requests were tested in the browser:

1. **404 Not Found**: Requesting a non-existent file
   URL: http://localhost:8080/nonexistent.html
2. **HTML with Image**: Accessing the main page with an embedded image
   URL: http://localhost:8080/index.html
3. **PDF File**: Downloading a PDF document
   URL: http://localhost:8080/sample-document.pdf
4. **PNG Image**: Viewing an image file
   URL: http://localhost:8080/images/library.png

4.4 Directory Listing

The server provides automatic directory listing for directories:

```
def serve_directory_listing(client_socket, dir_path, requested_path):
    try:
        entries = os.listdir(dir_path)
        entries.sort()

        html_content = f"""<!DOCTYPE html>
<html>
<head>
    <title>Directory listing for /{requested_path}</title>
    <style>
        body {{ font-family: monospace; }}
        a {{ text-decoration: none; color: blue; }}
        a:hover {{ text-decoration: underline; }}
        .directory {{ font-weight: bold; }}
        .file {{ }}
    </style>
```

```
</head>
<body>
    <h1>Directory listing for /{requested_path}</h1>
    <hr>
    <ul>
"""

        if requested_path != '.':
            parent_path = os.path.dirname(requested_path) if requested_path != '.' else ''
            html_content += f'        <li><a href="/{parent_path}" class="directory">../</a></li>\n'

        for entry in entries:
            entry_path = os.path.join(dir_path, entry)
            if requested_path == '.':
                url_path = entry
            else:
                url_path = f"{requested_path}/{entry}"

            if os.path.isdir(entry_path):
                html_content += f'        <li><a href="/{url_path}/" class="directory">{entry}/</a></li>\n'
            else:
                html_content += f'        <li><a href="/{url_path}" class="file">{entry}</a></li>\n'

        html_content += """    </ul>
    <hr>
</body>
</html>"""

        send_response(client_socket, 200, "text/html", html_content)
        print(f"Served directory listing: {dir_path}")

    except Exception as e:
        print(f"Error creating directory listing for {dir_path}: {e}")
        send_error_response(client_socket, 500, "Internal Server Error")
```

Example URL for directory listing: http://localhost:8080/books/

4.5 Using the HTTP Client

The HTTP client is used to fetch resources from the server:

```
# Fetch HTML content
python client.py localhost 8080 / downloads

# Download a PNG image
python client.py localhost 8080 /images/library.png downloads

# Download a PDF file
python client.py localhost 8080 /sample-document.pdf downloads

# View a directory listing
python client.py localhost 8080 /books/ downloads
```

Output from client when downloading a PNG file:

```
Status: HTTP/1.1 200 OK
Content-Type: image/png
File saved to: downloads/library.png
File type: image/png
File size: 245786 bytes
```

4.6 Network Setup for External Access

To make the server accessible to others on the local network:

1. Found the local IP address using:

   ```
   ipconfig

   ```

   My local IP address is 192.168.8.8

2. Allowed incoming connections through Windows Firewall:

   ```
   New-NetFirewallRule -DisplayName "HTTP File Server" -Direction
     Inbound -LocalPort 8080 -Protocol TCP -Action Allow

   ```

3. Verified the server was accessible at:

   ```
   http://192.168.8.8:8080/

   ```

4. Friends on the same network could access the server using this IP address

5 Conclusion

This laboratory work successfully implemented an HTTP file server using TCP sockets in Python and containerized it with Docker. Key achievements:

1. Created a functional HTTP server that handles GET requests
2. Implemented support for HTML, PNG, and PDF files
3. Added directory listing functionality for browsing content
4. Created a client application for accessing the server
5. Successfully deployed the application in a Docker container
6. Configured network access for local network testing

The project demonstrates the principles of HTTP communication, file serving, containerization, and network configuration. The server implements basic security measures like path traversal prevention and limited file type support.